# Experiment Title: State Space Search Problems

## Aim:

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem.
  Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

## Theory:

A. 2-Water Jug Problem

The 2-Water Jug Problem is a classic problem in AI. We have two jugs of fixed capacities and the goal is to get a specific amount of water in one of the jugs.

- Each state is written as (x, y):
  - x - water in Jug 1
  - y - water in Jug 2

Allowed moves:

- Fill a jug completely
- Empty a jug
- Pour water from one jug to the other until one is full or the other is empty

In the manual version, the user gives each move, and the program checks the current water levels after every step until the goal is reached.

B. Missionaries and Cannibals Problem

This problem is about safely moving 3 missionaries and 3 cannibals across a river using a boat that holds at most 2 people.

Rules:

- Missionaries can't be outnumbered by cannibals on either bank (or they'll get eaten)
- The boat cannot cross empty
- The boat can carry 1 or 2 people

State representation:

(M_left, C_left, Boat_position, M_right, C_right)

- M_left, C_left - number of missionaries/cannibals on the left bank
- M_right, C_right - number on the right bank
- Boat_position - 1 if on left bank, 0 if on right

## Algorithm / Methodology:

A. 2-Water Jug Problem (User-driven)

1. Start the program
2. Input capacities of Jug1 and Jug2
3. Set initial state (0, 0) and goal state
4. Show possible moves: fill, empty, pour Jug1 to Jug2, pour Jug2 to Jug1
5. Ask the user to choose a move
6. Update the jug state based on the move
7. Display the new state
8. Check if the goal is reached:
    - Yes - Stop
    - No - Repeat step 5

9. End

B. Missionaries and Cannibals Problem (Using BFS)

1. Represent state as (m1, c1, b, m2, c2)
2. Check that missionaries are never outnumbered on any bank
3. List all allowed moves: (1M), (2M), (1C), (2C), (1M1C)
4. Generate next states based on boat position and valid moves
5. Use Breadth-First Search (BFS) starting from (3,3,1,0,0)
6. Keep track of visited states to avoid repeating
7. Store the path
8. Stop when goal (0,0,0,3,3) is reached
9. Output the path of moves

## Dataset Description:

A. 2-Water Jug Problem

- Source: User input during execution of program
- State representation: (x, y); water in Jug1 and Jug2
- Size: Maximum states = (Jug1 capacity + 1) × (Jug2 capacity + 1) (+1 for empty state)
    - Example: Jug1 = 4L, Jug2 = 3L; 5 × 4 = 20 possible states

B. Missionaries and Cannibals Problem

- Source: Program generated states during execution of program
- State representation: (m1, c1, b, m2, c2)
- Size: Total possible states = 4 × 4 × 2 × 4 × 4 = 128 (only valid states are considered)

## Implementation:

2-Water Jug Problem

Code:

```
def water_jug_problem(capX, capY, target):

    x, y = 0, 0  # initial state: both jugs empty

    print("1 - Fill Jug X")

    print("2 - Fill Jug Y")

    print("3 - Empty Jug X")

    print("4 - Empty Jug Y")

    print("5 - Pour Jug X -> Jug Y")

    print("6 - Pour Jug Y -> Jug X")

    print("0 - Exit")

    while True:

        print(f"Current state - Jug A: {x}L, Jug B: {y}L")

        if x == target or y == target:

            print("Goal Reached!!")

            break

        move = int(input("Enter your move: "))

        if move == 1:

            x = capX

        elif move == 2:

            y = capY

        elif move == 3:

            x = 0

        elif move == 4:

            y = 0

        elif move == 5:
```

```
            transfer = min(x, capY - y)

            x -= transfer

            y += transfer

        elif move == 6:

            transfer = min(y, capX - x)

            y -= transfer

            x += transfer

        elif move == 0:

            print("Exiting!")

            break

        else:

            print("Invalid move")

# Example call

water_jug_problem(4, 3, 2)
```

## Output(A):

```
1 - Fill Jug X
2 - Fill Jug Y
3 - Empty Jug X
4 - Empty Jug Y
5 - Pour Jug X -> Jug Y
6 - Pour Jug Y -> Jug X
0 - Exit
Current state - Jug A: 0L, Jug B: 0L
Enter your move: 1
Current state - Jug A: 4L, Jug B: 0L
Enter your move: 5
Current state - Jug A: 1L, Jug B: 3L
Enter your move: 4
Current state - Jug A: 1L, Jug B: 0L
Enter your move: 5
Current state - Jug A: 0L, Jug B: 1L
Enter your move: 1
Current state - Jug A: 4L, Jug B: 1L
Enter your move: 5
Current state - Jug A: 2L, Jug B: 3L
Goal Reached!!
```

## Result Analysis(A):

This program helps us step-by-step to get exactly 2 litres of water. It shows how much water is in each jug after every move so we know what's happening. By filling, pouring, and emptying the jugs carefully, we finally get the right amount. At the end, it tells us the goal is reached.

B. Missionaries and Cannibals Problem

Code:

```
# Missionaries and Cannibals Problem

def is_valid(state): # This contains all the rules of game

  m1, c1, b, m2, c2 = state

  return 0 <= m1 <= 3 and 0 <= c1 <= 3 and 0 <= m2 <= 3 and 0 <= c2 <= 3
and ((m1 == 0 or m1 >= c1) and (m2 == 0 or m2 >= c2))

def generate_next_states(state):

  m1, c1, b, m2, c2 = state

  moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)] # Possible moves

  next_states = []

  for dm, dc in moves:

    # for boat to not carry more people than existing on that side

    if b == 1 and (dm > m1 or dc > c1):

      continue

    if b == 0 and (dm > m2 or dc > c2):

      continue

    if b == 1: # b == 1 means boat is on left side, b == 0 means boat is on
right side

      new_state = (m1 - dm, c1 - dc, 0, m2 + dm, c2 + dc)

    else:

      new_state = (m1 + dm, c1 + dc, 1, m2 - dm, c2 - dc)

    if is_valid(new_state):
```

```
      next_states.append(new_state)
  return next_states
def solve_bfs(start_state):
    queue = [(start_state, [start_state])]
    visited = set([start_state]) # To avoid revisiting states
    while queue:
        curr_state, path = queue.pop(0)
        if curr_state == (0, 0, 0, 3, 3):
            return path
        for next_state in generate_next_states(curr_state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None
start_state = (3, 3, 1, 0, 0)
solution = solve_bfs(start_state)
if solution:
  print("Solution path:")
  for step, state in enumerate(solution):
    m1, c1, b, m2, c2 = state
    boat_side = "Left" if b == 1 else "Right"
    print(
        f"Step {step + 1}: "
        f"Left = ({m1}M, {c1}C), "
        f"Right = ({m2}M, {c2}C), "
        f"Boat on {boat_side}"
    )
else:
```

```
print ("No solution found.")
```

## Output(B):

```
Solution path:
Step 1: Left = (3M, 3C), Right = (0M, 0C), Boat on Left
Step 2: Left = (3M, 1C), Right = (0M, 2C), Boat on Right
Step 3: Left = (3M, 2C), Right = (0M, 1C), Boat on Left
Step 4: Left = (3M, 0C), Right = (0M, 3C), Boat on Right
Step 5: Left = (3M, 1C), Right = (0M, 2C), Boat on Left
Step 6: Left = (1M, 1C), Right = (2M, 2C), Boat on Right
Step 7: Left = (2M, 2C), Right = (1M, 1C), Boat on Left
Step 8: Left = (0M, 2C), Right = (3M, 1C), Boat on Right
Step 9: Left = (0M, 3C), Right = (3M, 0C), Boat on Left
Step 10: Left = (0M, 1C), Right = (3M, 2C), Boat on Right
Step 11: Left = (1M, 1C), Right = (2M, 2C), Boat on Left
Step 12: Left = (0M, 0C), Right = (3M, 3C), Boat on Right
```

## Result Analysis(B):

The solution shows a step-by-step safe crossing of missionaries and cannibals from left to right. At each step, the number of missionaries is never less than the number of cannibals on either side, so no one gets eaten. The boat moves back and forth, carrying one or two people at a time, ensuring the rules are followed. By Step 12, all missionaries and cannibals have safely reached the right bank, solving the problem successfully.

## Conclusion:

**2-Water Jug Problem:**

By choosing moves step-by-step, you learn how to keep track of different situations and work toward a goal. It helps understand how problems can be solved by changing states carefully.

**Missionaries and Cannibals Problem:**

Using BFS and state validation teaches how to systematically explore possible solutions while respecting constraints. It strengthens skills in state-space search, avoiding invalid states, and finding the shortest safe path.